

Tracing Manual - Course Prerequisite Planner

Dataset yang Dipakai

Tracing memakai data dari:

```
data/courses.csv  
data/prerequisites.csv
```

Jumlah data:

Jenis Data	Jumlah
Mata kuliah/node	70
Relasi prasyarat/edge	40

1. Tracing Tree - Search Prefix pada Trie

Skenario

User mencari mata kuliah dengan prefix:

```
ET2343
```

Trie sudah berisi indeks dari kode dan nama semua mata kuliah. Setiap course dimasukkan dua kali:

1. Berdasarkan kode mata kuliah.
2. Berdasarkan nama mata kuliah.

Contoh insert kode:

```
trie.insert("ET234301", "ET234301");
```

Data yang Relevan

Kode mata kuliah yang memiliki prefix **ET2343**:

Kode	Nama
ET234301	Ethical Hacking dan Uji Keamanan Siber
ET234302	Keamanan Jaringan Komputer
ET234303	Internet of Things

Kode	Nama
ET234304	Komunikasi Profesional
ET234305	Pemrograman Web
ET234306	Komunikasi Data dan Jaringan Komputer

Proses Search

Input prefix dinormalisasi:

```
ET2343 -> et2343
```

Traversal Trie:

Langkah	Karakter	Node yang Dituju	Status
1	e	root -> e	Ada
2	t	e -> t	Ada
3	2	t -> 2	Ada
4	3	2 -> 3	Ada
5	4	3 -> 4	Ada
6	3	4 -> 3	Ada

Setelah node prefix **et2343** ditemukan, program mengambil **courseCodes** yang tersimpan pada node tersebut.

Hasil

```
ET234301
ET234302
ET234303
ET234304
ET234305
ET234306
```

Jumlah hasil:

```
6
```

Integrasi Tree dan Graph

Setelah Trie mengembalikan kode mata kuliah, program mengambil detail course dari **CourseGraph**:

```
Course course = graph.getCourse(code);
```

Program juga mengambil:

- 1. Jumlah prasyarat langsung dari **reverseAdjacencyList**.
- 2. Jumlah mata kuliah lanjutan dari **adjacencyList**.

Contoh ringkasan:

Kode	Prasyarat Langsung	Mata Kuliah Lanjutan
ET234301	1	1
ET234302	0	3
ET234303	2	2
ET234304	0	0
ET234305	2	4
ET234306	1	2

Kesimpulan tracing Tree: Trie berhasil menemukan semua course dengan prefix **ET2343** tanpa melakukan scan seluruh dataset secara manual.

2. Tracing Graph - Topological Sort dengan Kahn's Algorithm

Skenario

Tracing dilakukan pada subgraph yang relevan dengan jalur menuju Tugas Akhir.

Node:

Kode	Nama
ET234103	Algoritma dan Teknik Pemrograman
ET234203	Struktur Data dan Pemrograman Berorientasi Objek
ET234305	Pemrograman Web
ET234602	Pengembangan Sistem/Aplikasi (Capstone Project)
ET234702	Pra-TA / Metodologi Penulisan Ilmiah
ET234801	Tugas Akhir

Edge:

Prasyarat	Mata Kuliah Tujuan
-----------	--------------------

Prasyarat	Mata Kuliah Tujuan
ET234103	ET234203
ET234103	ET234305
ET234203	ET234305
ET234305	ET234602
ET234602	ET234801
ET234702	ET234801

Arah edge tetap:

```
prerequisite_code -> course_code
```

Inisialisasi Indegree

Node	Indegree	Alasan
ET234103	0	Tidak punya prasyarat pada subgraph
ET234702	0	Tidak punya prasyarat pada subgraph
ET234203	1	Masuk dari ET234103
ET234305	2	Masuk dari ET234103 dan ET234203
ET234602	1	Masuk dari ET234305
ET234801	2	Masuk dari ET234602 dan ET234702

Priority queue awal berisi semua node dengan indegree 0. Jika beberapa node sama-sama indegree 0, program memprioritaskan semester rekomendasi yang lebih kecil, lalu kode mata kuliah.

```
Queue = [ET234103, ET234702]
Order = []
```

Iterasi 1

Dequeue:

```
ET234103
```

Masukkan ke hasil:

```
Order = [ET234103]
```

Kurangi indegree tetangga keluar:

Tetangga	Indegree Lama	Indegree Baru
ET234203	1	0
ET234305	2	1

Karena ET234203 menjadi 0, masukkan ke queue. ET234203 berada di depan ET234702 karena semester 2 lebih kecil dari semester 7.

```
Queue = [ET234203, ET234702]
```

Iterasi 2

Dequeue:

```
ET234203
```

Masukkan ke hasil:

```
Order = [ET234103, ET234203]
```

Kurangi indegree tetangga keluar:

Tetangga	Indegree Lama	Indegree Baru
ET234305	1	0

Masukkan ET234305 ke queue:

```
Queue = [ET234305, ET234702]
```

Iterasi 3

Dequeue:

```
ET234305
```

Masukkan ke hasil:

```
Order = [ET234103, ET234203, ET234305]
```

Kurangi indegree tetangga keluar:

Tetangga	Indegree Lama	Indegree Baru
ET234602	1	0

Masukkan ET234602 ke queue:

```
Queue = [ET234602, ET234702]
```

Iterasi 4

Dequeue:

```
ET234602
```

Masukkan ke hasil:

```
Order = [ET234103, ET234203, ET234305, ET234602]
```

Kurangi indegree tetangga keluar:

Tetangga	Indegree Lama	Indegree Baru
ET234801	2	1

Belum ada node baru dengan indegree 0.

```
Queue = [ET234702]
```

Iterasi 5

Dequeue:

```
ET234702
```

Masukkan ke hasil:

```
Order = [ET234103, ET234203, ET234305, ET234602, ET234702]
```

Kurangi indegree tetangga keluar:

Tetangga	Indegree Lama	Indegree Baru
ET234801	1	0

Masukkan **ET234801** ke queue:

```
Queue = [ET234801]
```

Iterasi 6

Dequeue:

```
ET234801
```

Masukkan ke hasil:

```
Order = [ET234103, ET234203, ET234305, ET234602, ET234702, ET234801]  
Queue = []
```

Queue kosong, algoritma berhenti.

Hasil Topological Sort Subgraph

1. ET234103
2. ET234203
3. ET234305
4. ET234602
5. ET234702
6. ET234801

Urutan tersebut valid karena setiap mata kuliah selalu muncul setelah prasyaratnya.

3. Tracing Edge Case - Relasi Ditolak karena Cycle

Skenario

User mencoba menambahkan relasi:

```
ET234801 -> ET234103
```

Artinya Tugas Akhir dijadikan prasyarat untuk Algoritma dan Teknik Pemrograman.

Kondisi Graph Sebelum Edge Ditambahkan

Pada dataset sudah ada jalur:

```
ET234103 -> ET234203 -> ET234305 -> ET234602 -> ET234801
```

Jika edge baru ET234801 -> ET234103 diterima, maka akan terbentuk cycle:

```
ET234103 -> ET234203 -> ET234305 -> ET234602 -> ET234801 -> ET234103
```

Proses di addEdge

- 1. Program memvalidasi bahwa ET234801 dan ET234103 ada di courses.
- 2. Program memastikan source dan destination tidak sama.
- 3. Program menambahkan edge sementara ke adjacency list.
- 4. Program menjalankan hasCycle().
- 5. DFS cycle detection menemukan node yang sedang dalam status visiting.
- 6. Program menghapus kembali edge sementara.
- 7. Program melempar error dan relasi tidak disimpan.

Tracing Status DFS

Status:

```
0 = unvisited
1 = visiting
2 = visited
```

Salah satu jalur DFS yang menunjukkan cycle:

Langkah	Node	Status Saat Masuk	Edge Berikutnya
1	ET234103	1	ET234203
2	ET234203	1	ET234305
3	ET234305	1	ET234602

Langkah	Node	Status Saat Masuk	Edge Berikutnya
4	ET234602	1	ET234801
5	ET234801	1	ET234103

Pada langkah 5, DFS menemukan ET234103 yang masih berstatus 1 atau visiting. Ini berarti graph memiliki cycle.

Output Program

```
[ERROR] Gagal menambah relasi: Relasi menyebabkan cycle dan dibatalkan:
ET234801 -> ET234103
```

Kondisi Setelah Ditolak

Jumlah edge tetap:

```
40
```

Status graph tetap:

```
Tidak ada siklus
```

Kesimpulan tracing edge case: program tidak hanya mendeteksi cycle, tetapi juga menjaga konsistensi graph dengan melakukan rollback edge yang bermasalah.

Kesimpulan Tracing

- 1. Trie berhasil menjalankan search prefix dengan menelusuri karakter prefix, bukan scan manual seluruh dataset.
- 2. Topological sort menghasilkan urutan valid karena node hanya diproses setelah indegree menjadi 0.
- 3. Cycle detection berhasil menolak relasi yang membuat konflik prasyarat dan mengembalikan graph ke kondisi aman.